Optimization Algorithm in `poLCA`

This document translates the core optimization routine of the `poLCA` package (latent class analysis with polytomous outcomes) into mathematical notation. The implementation lives in `R/poLCA.R`, `src/poLCA.c`, and the surrounding R wrappers (`R/poLCA.postClass.C.R`, `R/poLCA.probHat.C.R`, `R/poLCA.dLL2dBeta.C.R`, `R/poLCA.updatePrior.R`).

The algorithm is a **hybrid EM / Newton-Raphson** procedure: - The **response probabilities** ρ are updated via a standard closed-form **EM M-step**. - The **class membership probabilities** π (or, equivalently, the multinomial logit coefficients β that parameterize them) are updated with a **single Newton-Raphson step** on the observed log-likelihood.

Why Hybrid?

Pure EM guarantees monotonic likelihood increases, but convergence for the multinomial-logit coefficients β can be very slow. A full Newton-Raphson inner loop inside every M-step would be expensive. Taking a **single observed-information Newton step** per EM iteration strikes a middle ground: it is cheap, more aggressive than a pure EM gradient step, and still drives the overall procedure toward a local optimum (modulo the error trap described in §3).

1. Model and Likelihood

Notation

- $i = 1, \dots, N$: observations
- $j = 1, \dots, J$: manifest (observed) variables
- $r = 1, \dots, R$: latent classes
- $k = 1, \dots, K_j$: categories of manifest variable j
- $y_{ij} \in \{1, \dots, K_j\}$: observed response (with $y_{ij} = 0$ denoting missing; this encoding is applied in `R/poLCA.R` when `na.rm=FALSE`)
- $\mathbf{x}_i \in \mathbb{R}^S$: covariate vector (including an intercept in practice)

Parameters

1. **Class-conditional response probabilities**

$$\rho_{jrk} = \Pr(Y_{ij} = k \mid C_i = r)$$

with $\sum_{k=1}^{K_j} \rho_{jrk} = 1$ for each (j, r).

2. **Class membership probabilities (priors)** When covariates are present, they are modeled by a multinomial logit with class 1 as the reference:

$$\pi_{i1}(\beta) = \frac{1}{1 + \sum_{s=2}^{R} \exp(\mathbf{x}_i^\top \beta_s)},$$

$$\pi_{ir}(\beta) = \frac{\exp(\mathbf{x}_i^\top \beta_r)}{1 + \sum_{s=2}^{R} \exp(\mathbf{x}_i^\top \beta_s)}, \qquad r = 2, \dots, R.$$

Here $\beta = (\beta_2^\top, \dots, \beta_R^\top)^\top \in \mathbb{R}^{S(R-1)}$.

When there are **no covariates** ($S = 1$, $\mathbf{x}_i = 1$), the priors reduce to global mixing proportions π_r with $\sum_{r=1}^{R} \pi_r = 1$.

Observed-data log-likelihood

For individual i, the class-conditional likelihood is

$$L_{ir}(\rho) = \prod_{j=1}^{J} \prod_{k=1}^{K_j} \rho_{jrk}^{\mathbb{I}(y_{ij}=k)},$$

where missing values ($y_{ij} = 0$) are excluded from the product (see `ylik` in `src/poLCA.c`).

The observed log-likelihood is

$$\ell(\rho, \beta) = \sum_{i=1}^{N} \log \left(\sum_{r=1}^{R} \pi_{ir}(\beta)\, L_{ir}(\rho) \right).$$

(In the code, the C function `ylik` scales each class-conditional likelihood by `DBL_MAX` to prevent underflow. The R wrapper removes this scaling in the log domain, so the computed value is the exact log-likelihood with no residual offset.)

---

2. The Hybrid Algorithm

The main loop (in `R/poLCA.R`) performs the following steps until convergence (measured by the increase in ℓ falling below `tol`):

2.1 E-step: Posterior class membership probabilities

Compute the posterior weight for each observation and class:

$$w_{ir} = \Pr(C_i = r \mid \mathbf{y}_i) = \frac{\pi_{ir}(\beta)\, L_{ir}(\rho)}{\sum_{s=1}^{R} \pi_{is}(\beta)\, L_{is}(\rho)}.$$

Code: `poLCA.postClass.C` $\rightarrow$ C function `postclass` in `src/poLCA.c`.

2.2 M-step for response probabilities ρ

Update the class-conditional response probabilities by their conditional posterior means:

$$\rho_{jrk}^{\text{new}} = \frac{\sum_{i:y_{ij}=k} w_{ir}}{\sum_{i:y_{ij}>0} w_{ir}}.$$

Missing responses ($y_{ij} = 0$) do not contribute to either the numerator or the denominator.

Code: `poLCA.probHat.C` $\rightarrow$ C function `probhat` in `src/poLCA.c`.

2.3 M-step for class proportions π (no covariates)

If $S = 1$ (no covariates), the priors are updated as simple averages of the posteriors:

$$\pi_r^{\text{new}} = \frac{1}{N} \sum_{i=1}^{N} w_{ir}.$$

Code: `prior <- matrix(colMeans(rgivy), nrow=N, ncol=R, byrow=TRUE)` in `R/poLCA.R`.

2.4 M-step for multinomial logit coefficients β (with covariates)

When covariates are present ($S > 1$), the β parameters do **not** have a closed-form M-step. Instead, the algorithm takes **one Newton-Raphson step** on the observed log-likelihood, holding the current posteriors w_{ir} fixed from the E-step. This is a **GEM (Generalized EM)** strategy: a single step is cheaper than a full inner optimization and is sufficient to increase the observed log-likelihood at most iterations. Although w_{ir} are treated as constants during the derivative computation, the formulas below (via Louis's identity) yield the **exact** gradient and observed information of the true marginal log-likelihood at the current parameter values. The precise sense in which this step generalizes (and departs from) a standard EM M-step is explained in §2.5.

Gradient The gradient of the observed log-likelihood with respect to β_r ($r = 2, \dots, R$) is

$$\mathbf{g}_r = \frac{\partial \ell}{\partial \beta_r} = \sum_{i=1}^{N} \mathbf{x}_i \, (w_{ir} - \pi_{ir}).$$

Stacking all $(R-1)$ blocks gives $\mathbf{g} \in \mathbb{R}^{S(R-1)}$. Intuitively, each term is a *prediction error* pushing the model's predicted prior π_{ir} toward the posterior target w_{ir}, analogous to weighted logistic regression.

Code: Computed in C function `d2lldbeta2` (`src/poLCA.c`) and returned via `poLCA.dLL2dBeta.C`.

Observed Information Matrix (Negative Hessian) The C function computes the **observed information matrix** $\mathcal{I}_{\text{obs}} = -\nabla^2 \ell$, which satisfies the missing-information identity $\mathcal{I}_{\text{obs}} = \mathcal{I}_{\text{comp}} - \mathcal{I}_{\text{miss}}$ (Louis 1982). In words, the curvature of the observed log-likelihood equals the complete-data curvature minus the variance of the complete-data score due to uncertainty in latent class membership.

For classes $r, t \in \{2, \dots, R\}$: - **Diagonal blocks** ($r = t$):

$$[\mathcal{I}_{\text{obs}}]_{rr} = \sum_{i=1}^{N} \mathbf{x}_i \mathbf{x}_i^\top \Big[\pi_{ir}(1 - \pi_{ir}) - w_{ir}(1 - w_{ir}) \Big].$$

- **Off-diagonal blocks** ($r \neq t$):

$$[\mathcal{I}_{\text{obs}}]_{rt} = \sum_{i=1}^{N} \mathbf{x}_i \mathbf{x}_i^\top \Big[w_{ir} w_{it} - \pi_{ir} \pi_{it} \Big].$$

The C code stores only the lower-triangle of the symmetric matrix and then mirrors it.

Code: C function `d2lldbeta2` in `src/poLCA.c`.

> **Sign handling in R:** The C function returns the observed information `ret$hess` ($\mathcal{I}_{\text{obs}}$). The R wrapper stores the negation as `dd$hess = -ret$hess`, so `dd$hess` is the true Hessian $\nabla^2 \ell$. Consequently `ginv(-dd$hess) = ginv(\mathcal{I}_{\text{obs}})`, and the update $\beta \leftarrow \beta + \mathcal{I}_{\text{obs}}^{-1} \nabla \ell$ is the standard Newton step for maximization.

Newton-Raphson Update The coefficient vector is updated as

$$\beta^{\text{new}} = \beta^{\text{old}} + \mathcal{I}_{\text{obs}}(\beta^{\text{old}})^{-1} \, \mathbf{g}(\beta^{\text{old}}).$$

In matrix notation (as in the code, using a pseudo-inverse `ginv` for robustness):

$$\beta \leftarrow \beta + \big[-\mathbf{H}(\beta)\big]^{-1} \nabla \ell(\beta),$$

where $\mathbf{H}(\beta)$ is the Hessian of the log-likelihood.

Code:

```
dd <- poLCA.dLL2dBeta.C(rgivy, prior, x)
b  <- b + ginv(-dd$hess) %*% dd$grad   # Newton-Raphson step
prior <- poLCA.updatePrior(b, x, R)    # update pi_i(beta)
```

2.5 How the β M-step relates to standard EM

In a textbook EM algorithm the M-step for β would **maximize the Q-function** obtained from the E-step:

$$Q(\beta \mid \beta^{\text{old}}) = \sum_{i=1}^{N} \sum_{r=1}^{R} w_{ir} \log \pi_{ir}(\beta) \; + \; \text{const.}$$

Because the response probabilities ρ are held fixed during this step, the gradient of Q with respect to β is identical to the gradient of the **observed** log-likelihood:

$$\nabla_\beta \ell = \nabla_\beta Q = \sum_{i=1}^{N} \mathbf{x}_i \, (w_{ir} - \pi_{ir}).$$

So a simple gradient-ascent step on Q would also be a gradient-ascent step on ℓ. The algorithm, however, does **not** take a step on Q. It takes a **Newton step on the observed log-likelihood** using the *observed* information matrix $\mathcal{I}_{\text{obs}} = \mathcal{I}_{\text{comp}} - \mathcal{I}_{\text{miss}}$ (Louis 1982).

What a standard EM M-step would use	What the code actually uses
Objective: $Q(\beta)$	Objective: $\ell(\beta)$
Hessian: complete-data information $\mathcal{I}_{\text{comp}} = -\nabla^2 Q$	Hessian: observed information $\mathcal{I}_{\text{obs}} = -\nabla^2 \ell$
Guaranteed to increase Q (and therefore ℓ)	More aggressive; can overshoot and *decrease* ℓ

This is why the procedure is a **GEM** (Generalized EM) rather than pure EM: the β update is designed to increase the observed log-likelihood directly, not merely to maximize the surrogate Q. The posteriors w_{ir} are treated as fixed constants during the derivative computation, but because $\nabla \ell = \nabla Q$, they appear naturally in the gradient formula. The price of the more aggressive step is the possibility of overshoot, which is why §3 includes the likelihood-decrease error trap and random restarts.

---

3. Convergence and Error Handling

The log-likelihood is recomputed after each full pass:

$$\ell^{(t)} = \sum_{i=1}^{N} \log \left(\sum_{r=1}^{R} \pi_{ir}^{(t)} L_{ir}^{(t)} \right).$$

(The C code scales likelihoods by `DBL_MAX` and the R wrapper cancels that scaling, so the computed value is the exact log-likelihood.)

The iteration stops when

$$\Delta \ell = \ell^{(t)} - \ell^{(t-1)} < \texttt{tol}$$

or when `iter > maxiter`.

Error trap: Because a Newton-Raphson step can overshoot and decrease the likelihood (unlike a pure EM step), the code sets `error <- TRUE` if - `dll` is `NA`, or - $S > 1$ and $\Delta \ell < -10^{-7}$ (a small decrease is tolerated, but anything larger triggers a restart).

When an error is triggered, the model restarts from a new random set of initial response probabilities (`probs`). This is repeated inside the `while(error)` loop, and across `nrep` independent replications to mitigate local maxima.

---

4. Summary Pseudocode

```
Input: data y (N × J), covariates x (N × S), number of classes R
Initialize: random rho, beta = 0, pi_i(beta) from multinomial logit

repeat
    // E-step
    for each i, r:
        L_ir = prod_{j: y_{ij}>0} rho_{j,r,y_{ij}}   // in C: scaled by DBL_MAX to prevent underflow
        w_ir = pi_ir * L_ir / sum_s pi_is * L_is

    // M-step for rho
    for each j, r, k:
        rho_{jrk} = sum_{i: y_{ij}=k} w_ir / sum_{i: y_{ij}>0} w_ir

    // M-step for pi (or beta)
    if S == 1:
        pi_r = (1/N) sum_i w_ir
    else:
        // One Newton-Raphson step for beta
        g_r = sum_i x_i (w_ir - pi_ir)      for r = 2..R
        I_obs = as defined in §2.4
        beta ← beta + I_obs^{-1} g
        pi_i ← poLCA.updatePrior(beta, x, R)   // recompute multinomial logits

    ell ← compute log-likelihood
until ell increase < tol or maxiter reached
```

---

5. Key Takeaways

Component	Method	Location
Posteriors w_{ir}	Standard E-step	`src/poLCA.c::postclass`
Response probs ρ	Closed-form EM M-step	`src/poLCA.c::probhat`
Mixing proportions (no covariates)	Closed-form EM M-step	`R/poLCA.R`
Covariate coefficients β	**Newton-Raphson** on observed log-likelihood	`src/poLCA.c::d2lldbeta2` + `R/poLCA.R`
Prior update $\pi_i(\beta)$	Multinomial logit	`R/poLCA.updatePrior.R`

Because the β update uses the **observed information matrix** (which accounts for the uncertainty in the latent class membership), the step is more aggressive than a pure EM gradient step. The price is that it is not guaranteed to be an ascent step at every iteration, which is why the code includes the likelihood-decrease error trap and multiple random restarts (`nrep`).